DAY 08 · SPEC-DRIVEN

Code from a Spec

Turning a written spec into correct encoders and parsers.

3 hours / day08_spec_driven / Spec implementation

INSIDE THIS BOOKLET

Specs Parsers Encodings Round-trips

How to use this booklet

Each day is a working session, not a reading assignment. This booklet is the typeset version of the day's `README.md`: keep the folder open, type the examples into a Python REPL, and run the tests as you go. The *Exam lens* and end-of-day review are study aids added to the original material.

Read & type
Type every example. Predict the output first.

Practice
Implement or debug one test at a time.

Test
Run the suite. Read the first failing line.

Log
Record each bug in `bug_journal.md`.

You will need: Python 3.10+ · a terminal · `day08_spec_driven/` · the course runner `python run.py 8`

In this session

Why encoders and decoders? 3

1 How to read a spec (do this on exam day too) 3

2 Two ways to write a decoder 4

3 Storing integers as bytes 5

4 Base64: bytes as safe text 7

5 Percent-encoding: making text safe for URLs 8

6 Bencode: nested data, BitTorrent style 8

7 The toolbox: Python you'll meet in today's exercises 9

Exercises 11

End-of-day checklist 12

> **◎ Today's goal**
>
> Turn a written spec into correct code fast. In the exam the spec describes an algorithm and the code claims to implement it; your job is to make the code match. Today you build four encoders/decoders from a spec, which trains the two skills you need: **reading specs precisely** and **byte-level parsing**.

Why encoders and decoders?

An **encoder** turns some data (numbers, text, lists) into a sequence of bytes or characters, following a set of rules called a **format**. A **decoder** reads that format and rebuilds the original data. UTF-8 from Day 5 is one example: `.encode()` and `.decode()`. They make ideal practice for this exam, for three reasons:

1. **Their specs are precise down to the last bit**, so "almost right" is simply wrong. That's exactly how the exam is graded: a test either passes or it doesn't.
2. **Every decoder is a parsing loop**: a position that moves through the input, reading one piece at a time. That's the same structure as the UTF-8 decoder (Day 5), the tokenizer (Day 9) and the snapshot reader (Day 10).
3. **They come in pairs**, so you can always test one against the other.

The four formats today are all real and widely used: **base64** (email attachments, images embedded in web pages), **varints** (Google's Protocol Buffers, WebAssembly), **percent-encoding** (every URL you've ever seen), and **bencode** (BitTorrent).

1 How to read a spec (do this on exam day too)

A spec is a list of promises the code must keep. A planted bug usually breaks *one specific promise* while keeping the overall idea intact. So if you read the spec loosely ("it encodes stuff in base64, fine"), you'll miss exactly the detail that's wrong. Reading precisely means turning every sentence into something you could check.

1. **Read it all once, fast.** Get the shape: inputs, outputs, main steps.
2. **Second pass with a pen:** turn every sentence into a checklist item. Underline the words that change behaviour: *inclusive, exclusive, first, last, ties, at most, at least, sorted, stable, case-insensitive, bytes, characters, UTF-8, big-endian, padding, raise, ignore, skip, empty*.
3. **Write tiny examples by hand** for each rule (e.g. "`'é'` → `C3 A9` → `w6k=`"). These are your mental test cases.
4. **Map each checklist item to code** (in the exam: to the line that is supposed to implement it). A checklist item with no matching code, or code with no matching item, is a suspect.
5. **The tests are the final word.** Where the spec is vague, read the test that covers it.

1.1 An example of step 2

Here's one sentence from today's bencode spec: > *"keys are encoded as byte strings and SORTED BY THEIR RAW BYTES … Raise ValueError for … unsorted or duplicate dict > keys."* Broken into checkable items, that's:

- ☐ the encoder turns every key into bytes;
- ☐ the encoder sorts the keys;
- ☐ the sort compares **bytes**, not the original strings;
- ☐ the decoder checks that keys arrive in sorted order, and raises `ValueError` if not;
- ☐ the decoder raises `ValueError` for a duplicate key.

Five items from one sentence. In exam code, each should correspond to a line. If you can't find the line that checks for duplicates, you've probably found a bug, without running a single test.

2 Two ways to write a decoder

Every decoder you'll meet has one of two shapes. Recognising which one you're looking at tells you where the bugs are likely to be.

2.1 Shape 1: cursor-based parsing

> **Cursor-based parsing**
>
> **Cursor-based parsing** — every decoder in this course uses this shape:

```
def decode(data):
    pos = 0
    def need(n):                      # helper: are there n more bytes?
        if pos + n > len(data):
            raise ValueError("truncated")
    out = []
    while pos < len(data):
        ...read a header at data[pos]...
        pos += consumed               # ALWAYS advance
    return out
```

The **cursor** is `pos`, the position of the next byte that hasn't been read yet. Each time round the loop, the decoder looks at the data at `pos`, works out what kind of item starts there and how long it is, reads it, and moves `pos` past it. It's like reading a sentence with your finger under the current word. Two parts of this shape deserve attention:

- **The `need(n)` helper** puts the question "are there at least `n` bytes left?" in one place, and every read calls it first. If a read skips this check, slicing past the end silently returns *fewer* bytes than asked for (slices never raise, Day 1), and the decoder carries on with bad data instead of raising the `ValueError` the spec requires. That's a very common planted bug.
- **"Always advance."** If any branch forgets to move `pos` forward, the loop reads the same item forever.

2.2 Shape 2: recursive descent

> **Recursive descent**
>
> **Recursive descent** for nested formats (JSON, bencode): a function `parse_value(data, pos) -> (value, new_pos)` that dispatches on the first byte, calling itself for nested values. Returning the new position is the key idea.

Some formats can nest: a list can contain lists, which contain dicts, which contain more lists... A flat loop can't handle that, because you don't know in advance how deep the nesting goes. The solution is a function that **calls itself**. That's called *recursion*. `parse_value(data, pos)` looks at the byte at `pos` and decides what kind of value starts there. If it's a simple value (a number), it reads it. If it's a list, it reads the list's items by calling `parse_value` again for each one, and an item might itself be a list, which calls `parse_value` again, and so on. The nesting in the data is mirrored by the nesting of the function calls. The crucial detail is that **every call returns two things: the value it read, and the position where it stopped**. The caller needs that position to know where the next item starts:

```
def parse_list(data, pos):          # data[pos] is the opening "l"
    pos += 1                        # skip the "l"
    items = []
    while data[pos:pos+1] != b"e":  # until the closing "e"
        item, pos = parse_value(data, pos)   # read one item AND move past it
        items.append(item)
    return items, pos + 1           # skip the "e"
```

A classic bug is to ignore the returned position (`item, _ = parse_value(data, pos)`), so the same item is read again and again.

2.3 The round-trip property

> **Round-trip property**
>
> For any encoder/decoder pair, `decode(encode(x)) == x` for all valid x. Easy to test with random inputs — use it in your scratch scripts.

If you encode something and then decode the result, you must get back exactly what you started with. This gives you a free test without needing to know the correct encoded output: generate random inputs, round-trip them, and compare (the differential testing from Day 6). It doesn't prove the encoding follows the format (a wrong-but-consistent pair would also round-trip), but it catches most decoder bugs instantly.

3 Storing integers as bytes

Binary formats constantly need to store integers: lengths, counts, ids. There are two approaches.

3.1 Fixed width: always the same number of bytes

The simplest approach is to decide that a number always takes, say, 4 bytes. The decoder knows to read exactly 4 bytes. Python does the conversion with `to_bytes` and `from_bytes` (Day 5):

```
>>> (300).to_bytes(4, "big")       # 300 = 0x012C
b'\x00\x00\x01,'
>>> int.from_bytes(b"\x00\x00\x01\x2c", "big")
300
```

(The `,` is just how Python displays the byte 0x2C, which happens to be the comma character.) To see what `to_bytes` does inside: the byte at position `k` (counting from the low end) is `(n >> (8 * k)) & 0xFF`, i.e. shift that byte down to the bottom and keep 8 bits. For example `(300 >> 8) & 0xFF` is 1, the second byte. The weakness of fixed width is waste: storing the number 5 in 4 bytes means three of them are just zeros.

3.2 Variable width: varints (LEB128)

A **varint** ("variable-length integer") uses fewer bytes for small numbers and more for big ones. Small numbers are far more common in practice (lengths, small ids), so this saves a lot of space. The format used here is called **LEB128**. The rules:

- Cut the number into **groups of 7 bits**, starting from the low end.
- Each group goes into one byte, **lowest group first**.
- The 8th bit (the top bit, value `0x80`) of each byte is a **flag**: 1 means "more bytes follow", 0 means "this is the last byte".

Varint / LEB128 (Protocol Buffers, WebAssembly, many token formats): 7 bits of payload per byte, least-significant group first, high bit `0x80` = "more bytes follow".

```
300 = 0b1_0010_1100
groups of 7 (low first): 010_1100, 000_0010
bytes: 0xAC (0x2C | 0x80, more follow), 0x02 (last)
```

Let's go through that example slowly:

1. 300 in binary is `100101100` (256 + 32 + 8 + 4).
2. Cut it into 7-bit groups from the right: the low group is `0101100` (= 0x2C = 44), and what's left is `10` (= 2).
3. The first byte holds the low group, with the "more follows" flag set: `0x2C | 0x80` = `0xAC`.
4. The second byte holds the remaining group, with the flag off because it's the last: `0x02`.

So 300 is stored as the two bytes `AC 02`. Any number below 128 fits in a single byte, with the flag off. **Encoding**, as a procedure: while the number is 128 or more, output its low 7 bits (`n & 0x7F`) with the flag set (`| 0x80`), then drop those 7 bits (`n >>= 7`). When what's left is below 128, output it as the final byte, without the flag. A `bytearray` (Day 5) is the natural place to collect the output. **Decoding** reverses it. Keep a running `result` (starting at 0) and a `shift` (starting at 0). For each byte: take its low 7 bits, move them into place with `<< shift`, and OR them into `result`. Then, if the byte's flag is off, you're done. Otherwise add 7 to `shift` and read the next byte. Here's the trace for `AC 02`:

byte	low 7 bits	shifted into place	result	flag
`0xAC`	`0x2C` = 44	`44 << 0` = 44	44	on, so continue, shift = 7
`0x02`	`0x02` = 2	`2 << 7` = 256	44 + 256 = **300**	off, so stop

The `shift` grows because *later* bytes hold *higher* groups. That's what "least significant group first" means. If the data runs out while the flag still says "more follows", the varint is truncated, and the spec says to raise `ValueError`. Typical bugs: the groups put together in the wrong order (shifting the *result* instead of the new bits), forgetting to set or test `0x80`, or not noticing that the data ended in the middle of a varint.

3.3 Negative numbers: ZigZag

Varints only work for non-negative numbers. To store a negative number, you first map it to a non-negative one, in a way that keeps numbers *close to zero* small, whether they're positive or negative. **ZigZag** does this by interleaving them: ZigZag maps signed to unsigned so small negatives stay small: `0→0, -1→1, 1→2, -2→3, ...` `zz = (n << 1) ^ (n >> 63)` for 64-bit ints; in Python simply `2*n if n >= 0 else -2*n - 1`.

n	0	−1	1	−2	2	−3
zigzag(n)	0	1	2	3	4	5

Positive numbers go to even codes (`2n`) and negative numbers to odd codes (`-2n - 1`). To decode, look at whether the code is even or odd: `z // 2` if `z` is even, `-(z + 1) // 2` if it's odd. (The bit-trick formula is what you'd see in C code. In Python the plain arithmetic version is clearer.)

4 Base64: bytes as safe text

4.1 What it's for

Some places can only carry ordinary printable text: email bodies, JSON documents, URLs, HTML. Raw bytes (an image, say) contain values that would be misread as control characters or cut off. **Base64** writes any bytes using only 64 "safe" characters: `A–Z`, `a–z`, `0–9`, `+` and `/`. The price is size: the result is about a third bigger than the original.

4.2 How it works

Each base64 character can stand for 64 different values, which is exactly 6 bits ($2^6 = 64$). Bytes are 8 bits. The smallest whole number of bytes that splits evenly into 6-bit pieces is 3 bytes = 24 bits = 4 pieces. So base64 works on **groups of 3 bytes, turning each group into 4 characters**: Take bytes 3 at a time = 24 bits, split into four 6-bit numbers, map each to `A–Z a–z 0–9 + /`. If the last group has 1 byte: emit 2 chars + ==; 2 bytes: 3 chars + =.

```
"Man" = 4D 61 6E = 010011 010110 000101 101110 = 19 22 5 46 = T W F u -> "TWFu"
"Ma"  = 4D 61    = 010011 010110 0001(00)      = 19 22 4    = T W E = -> "TWE="
```

The first example, step by step:

1. The text `"Man"` is the three bytes `4D 61 6E`.
2. In binary that's `01001101 01100001 01101110`, 24 bits in a row.
3. Re-cut them into four groups of 6 bits: `010011 010110 000101 101110`, which are the numbers 19, 22, 5 and 46.
4. Look each number up in the alphabet. Positions 0–25 are `A–Z`, 26–51 are `a–z`, 52–61 are `0–9`, 62 is `+` and 63 is `/`. So 19 → `T`, 22 → `W`, 5 → `F`, 46 → `u`. The result is `"TWFu"`.

When the input length isn't a multiple of 3, the last group is short. With 2 bytes (16 bits), pad with zeros to 18 bits, which is three 6-bit pieces, write 3 characters, and add one = so the output length stays a multiple of 4. With 1 byte (8 bits), pad to 12 bits, write 2 characters, and add ==. You can check any example in the REPL with the standard library:

```
>>> import base64
>>> base64.b64encode(b"Ma")
b'TWE='
```

A strict decoder must reject an input whose length isn't a multiple of 4, characters outside the alphabet, and = in any place other than the last one or two positions.

5 Percent-encoding: making text safe for URLs

5.1 What it's for

URLs may only contain a limited set of ASCII characters. Spaces, accents, &, ?, / and emoji either aren't allowed or have special meanings. **Percent-encoding** replaces each such character with % followed by two hex digits. You've seen it in addresses like `.../search?q=caf%C3%A9`.

5.2 The rule

Encode the string to UTF-8; every byte that is not "unreserved" (`A–Z a–z 0–9 - . _ ~`) becomes `%XX` (uppercase hex). Decoding: turn `%XX` back into bytes, collect ALL bytes, then decode UTF-8 once. `"café"` → `caf%C3%A9`. The key point, which connects to Day 5: the escaping works on **bytes**, not characters. é is two bytes in UTF-8 (`C3 A9`), so it becomes *two* escapes, `%C3%A9`, not `%E9` (its code point). A space is the single byte 0x20, so it becomes `%20`. Decoding goes the other way: each `%XX` becomes one byte, and other characters are kept as their own bytes. Then **all** the bytes are decoded as UTF-8 *at the end*. `%C3` on its own isn't a character, only half of one, so decoding each escape separately would fail. It's the same "join the bytes first, then decode once" rule as for tokenizers. Typical bugs: escaping code points instead of bytes, lowercase hex digits (`%c3`) where the spec demands uppercase, and decoding each `%XX` separately.

6 Bencode: nested data, BitTorrent style

6.1 What it is

Bencode is a compact format for nested data: integers, byte strings, lists and dictionaries. It's similar in purpose to JSON, but much simpler to parse, because every value starts with a character that tells you its type. That makes it the perfect recursive-descent exercise.

6.2 The four kinds of value

- integer: `i42e`, `i-7e`, `i0e`. (`i-0e`, leading zeros like `i03e` are invalid)
- byte string: `<length>:<bytes>` → `4:spam`. **The length counts bytes**, so `"é"` is `2:\xc3\xa9`.
- list: `l<items>e` → `l4:spami42ee`
- dict: `d<key><value>...e`, keys are byte strings **sorted** by their raw bytes → `d3:cow3:moo4:spam4:eggse`

Each in more detail:

- **Integers** are written between `i` and `e`. The forbidden forms (`-0`, leading zeros) exist so that every number has exactly *one* valid encoding, the same idea as the ban on overlong sequences in UTF-8.

- **Byte strings** start with their length, then a colon, then exactly that many bytes. No closing marker is needed, because the length says where the string ends. This is called a **length-prefixed** string. It's why a string can contain *any* bytes, even `e` or `:`, without confusing the decoder. And the length counts bytes, not characters: that's Day 5 again.
- **Lists** are `l`, then the items one after another, then `e`.
- **Dicts** are `d`, then key, value, key, value, …, then `e`. The keys must appear sorted.

6.3 Decoding an example by hand

Let's decode `l4:spami42ee` the way a recursive-descent decoder would:

1. Position 0 is `l`: a list starts. Read items until an `e`.
2. Position 1 is `4`, a digit, so a byte string starts. Read the length up to the `:`, which is `4`, then take the next 4 bytes: `spam`. We're now at position 7.
3. Position 7 is `i`: an integer. Read up to the next `e`: `42`. We're now at position 11.
4. Position 11 is `e`: the list ends.

The result is `["spam", 42]`. Notice how each step started exactly where the previous one stopped. That's the returned position from section 2 at work.

7 The toolbox: Python you'll meet in today's exercises

7.1 A string as a lookup table: `ALPHABET[i]` and `ALPHABET.find(c)`

Base64 maps the numbers 0–63 to 64 characters. The simplest way to store that mapping is a 64-character string:

```
ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
ALPHABET[0]        # 'A'    number -> character: just index the string
ALPHABET.find("c") # 28     character -> number: find its position
ALPHABET.find("!") # -1     not in the alphabet
```

`find` returns `-1` for "not found" (Day 2, section 2), so a decoder checks `if idx < 0: raise ValueError(...)`. Using `-1` as an index without that check would silently read the *last* character, `"/"`.

7.2 Hex digits in text: `int(h, 16)` and `f"{b:02X}"`

`int` takes an optional second argument, the base: `int("ff", 16)` is 255, `int("101", 2)` is 5. That's how `%C3` in a URL becomes the byte `0xC3`. In the other direction, the format spec `02X` writes a number as **uppercase** hexadecimal with at least 2 digits: `f"%{195:02X}"` is `"%C3"`, and `f"{5:02X}"` is `"05"` (Day 2, section 4; lowercase `x` gives lowercase digits).

7.3 Padding and counting padding

Bytes have the same `ljust` as strings (Day 2): `b"ab".ljust(3, b"\x00")` is `b"ab\x00"`, which fills a short final group up to 3 bytes with zero bytes. To count how many `=` padding characters a Base64 string ends with, compare its length before and after stripping them:

```
pad = len(text) - len(text.rstrip("="))      # "QQ==" -> 4 - 2 = 2
```

7.4 Bytes and integers: `int.from_bytes` and `to_bytes`

Day 5 introduced these. `int.from_bytes(b"\x01\x00", "big")` reads the bytes as one big number (256 here), and `n.to_bytes(3, "big")` writes `n` as exactly 3 bytes. Base64 uses them to turn 3 bytes into one 24-bit number, cut it into four 6-bit pieces with shifts and masks (Day 5, section 3), and back.

7.5 `while True` with a `return` inside

`while True:` loops forever *on purpose*. The way out is a `return` (or `break`) inside the body, taken when the work is done. A varint encoder can't know in advance how many bytes it'll produce, so it emits one byte per round and returns after the byte that has no "more follows" bit. For each `while True` you read, find every exit and make sure one is always reached. The opposite technique, `for count in range(10):` followed by a `raise` after the loop, is a **safety limit**: a valid 64-bit varint is at most 10 bytes long, so if the loop runs out without finding the last byte, the data is invalid. `result |= piece << shift` is the "or-equals" shortcut (like `+=`): it switches on, in `result`, the bits of `piece` moved to position `shift`.

7.6 `bool` is a kind of `int`

`True` and `False` are, for historical reasons, the integers 1 and 0 in disguise: `isinstance(True, int)` is `True`, and `True + True` is `2`. So an encoder that handles integers with `if isinstance(value, int):` will happily encode `True` as `1`. When a spec says booleans aren't allowed, the bool check must come **before** the int check:

```
if isinstance(value, bool):
    raise TypeError("bool not supported")
if isinstance(value, int):
    ...
```

For error messages, `type(value).__name__` gives the name of a value's type as a string: `"float"` for `3.5`, `"set"` for `{1}`.

7.7 `isdigit()` is broader than you think

`isdigit()` accepts any Unicode digit, not only `0`–`9`: `"²".isdigit()` and `"٢٢".isdigit()` (Arabic-Indic digits) are both `True`, and `int("٢٢")` even converts the latter to 12. A strict format that only allows ASCII digits must also check `isascii()`, which is `True` when every character is plain ASCII (code point below 128):

```
if not digits.isdigit() or not digits.isascii():
    raise ValueError(...)
```

7.8 One byte as bytes: `data[pos:pos + 1]`

Indexing a `bytes` object gives a number, so `data[pos] == b"i"` is always `False` (`105` is not `b"i"`; Day 5, section 2). Decoders therefore compare a one-byte **slice**: `data[pos:pos + 1] == b"i"`. The slice also has a nice side effect: at the end of the data it gives `b""` instead of raising `IndexError`. To turn a few ASCII bytes into text that can be checked with string methods, decode them: `data[a:b].decode("ascii", "replace")`. The `"replace"` puts `␣` in place of any non-ASCII byte instead of raising, so the string check that follows is what reports the error.

7.9 "Try to decode, and fall back"

Bencode strings are raw bytes, but most of them are really text. The reference solution returns a `str` when the bytes are valid UTF-8, and leaves them as `bytes` otherwise:

```
try:
    return raw.decode("utf-8")
except UnicodeDecodeError:
    return raw
```

`try`/`except` (Day 4, section 7) is the natural way to ask "does this work?" when the only way to find out is to try it.

7.10 A function inside a function

Shape 1 above defines a small helper, `need(n)`, *inside* `decode`. A nested function can **read** the variables of the function around it, such as `pos` and `data`, which is why `need` doesn't take them as parameters. It can't **assign** to them without one extra declaration (`nonlocal`), which you'll meet on Day 10.

7.11 Set union: `UNRESERVED | set(safe)`

For sets, `|` means **union**: every element that's in either set (Day 3, section 8). `set("/:")` makes a set of the characters of a string, `{"/", ":"}`. So `UNRESERVED | set(safe)` is "the always-safe characters, plus the ones the caller added".

Exercises

Practice lab

Implement each function from the spec in its docstring. Work one file at a time; the tests are the final word.

File	What	Tests compare against
`exercises/ex01_base64.py`	base64 encode/decode, strict validation	Python's `base64` module
`exercises/ex02_varint.py`	LEB128 varints + zigzag + streams	hand-computed vectors, round trips
`exercises/ex03_percent.py`	percent-encode/decode with UTF-8	`urllib.parse`
`exercises/ex04_bencode.py`	full bencode encoder + recursive decoder with errors	hand-computed vectors

The last column tells you where the tests' expected answers come from. For base64 and percent-encoding, they come from Python's own standard library, so whenever you're unsure what the right output is, you can ask the REPL (`base64.b64encode(...)`, `urllib.parse.quote(...)`). For varints and bencode, the answers were worked out by hand from the spec, so try computing one or two yourself as practice.

Time-box each exercise

Each is ~30–60 lines. **Time-box each one to 30–40 minutes.** If you exceed it, read the solution, then close it and re-implement from memory.

```
python run.py 8 -k ex01
```

End-of-day checklist

Tick every box

- [ ] I can turn a spec sentence into a checklist item.
- [ ] My encoder/decoder passes the round-trip property: `decode(encode(x)) == x`.
- [ ] I can parse a length-prefixed format without losing the cursor.
- [ ] All day-8 tests pass.

Exam lens

- Turn every spec sentence into a checklist item; the tests win where the spec is vague.
- Cursor-based parsing: always advance, validate lengths, and return the new position.
- Round-trip tests catch encoder/decoder mismatches faster than reading the code.

Tonight's bug journal

Where	Symptom	Root cause / fix

One row per bug. Re-read this page on Day 10.